

NAME- Jiten Mourya

UID-23BCC70050

SUB- ADBMS

EXP-11

➤ **AIM:** To demonstrate how **row-level locking and transactions** can prevent duplicate enrollments and preserve **data consistency** when multiple users attempt concurrent operations on the same student record.

➤ **THEORY:**

- Transactions in DBMS: Transactions ensure a sequence of operations executes as a single unit, maintaining Atomicity, Consistency, Isolation, and Durability (ACID).
- Concurrency Problems: Without proper locking, two users could insert or update the same student record simultaneously, causing duplicate enrolments or inconsistent data.
- Unique Constraints: Define (student_name, course_id) as UNIQUE to prevent duplicate enrollments.
- Row-Level Locking with SELECT FOR UPDATE: Locks specific rows during a transaction, blocking other users from updating the same rows until the transaction is committed or rolled back.
- Prevents race conditions in concurrent environments.
- Locking Preserves Consistency: Ensures no duplicate enrollments occur.
- Conflicting operations are serialized automatically.

➤ **CODES:**

```
CREATE TABLE StudentEnrollments (  
    enrollment_id INT PRIMARY KEY,  
    student_name VARCHAR(100) NOT NULL,  
    course_id VARCHAR(10) NOT NULL,  
    enrollment_date DATE NOT NULL,  
    UNIQUE (student_name, course_id)  
);  
  
-- First, a valid enrollment  
INSERT INTO StudentEnrollments (enrollment_id,  
    student_name, course_id, enrollment_date)  
VALUES (1, 'Ashish', 'CSE101', '2024-07-01');  
  
-- Try to insert the same student and course again (should  
    fail)  
INSERT INTO StudentEnrollments (enrollment_id,  
    student_name, course_id, enrollment_date)  
VALUES (2, 'Ashish', 'CSE101', '2024-07-01');  
  
-- This will cause a UNIQUE constraint violation  
START TRANSACTION;  
  
SELECT * FROM StudentEnrollments WHERE student_name =  
    'Ashish' AND course_id = 'CSE101' FOR UPDATE;  
  
-- (Keep transaction open here to hold the lock)  
  
-- This query will wait (block) until User A finishes  
    (commits/rollbacks)  
  
UPDATE StudentEnrollments SET enrollment_date = '2024-07-  
    02'  
  
WHERE student_name = 'Ashish' AND course_id = 'CSE101';  
  
START TRANSACTION;  
  
SELECT * FROM StudentEnrollments WHERE enrollment_id = 1  
    FOR UPDATE;
```

```
UPDATE StudentEnrollments SET enrollment_date = '2024-07-03' WHERE enrollment_id = 1;
```

```
-- Do not commit yet
```

```
START TRANSACTION;
```

```
SELECT * FROM StudentEnrollments WHERE enrollment_id = 1  
FOR UPDATE;
```

```
UPDATE StudentEnrollments SET enrollment_date = '2024-07-04' WHERE enrollment_id = 1;
```

```
-- This will wait until User A commits or rollbacks,  
preserving consistency
```

```
COMMIT; -- For User A
```

```
COMMIT; -- For User B
```

```
SELECT * FROM StudentEnrollments WHERE enrollment_id =  
1;
```

OUTPUTS:

The screenshot shows a web-based MySQL IDE interface. The top navigation bar includes a search bar and a user profile icon labeled 'JITEN MOURYA'. The main area displays a SQL script with the following queries:

```
14  
15 START TRANSACTION;  
16 SELECT * FROM StudentEnrollments WHERE student_name = 'Ashish' AND course_id = 'CSE101' FOR UPDATE;  
17  
18 UPDATE StudentEnrollments SET enrollment_date = '2024-07-02'  
19 WHERE student_name = 'Ashish' AND course_id = 'CSE101';  
20 START TRANSACTION;  
21 SELECT * FROM StudentEnrollments WHERE enrollment_id = 1 FOR UPDATE;  
22 UPDATE StudentEnrollments SET enrollment_date = '2024-07-03' WHERE enrollment_id = 1;  
23 -- Do not commit yet  
24 START TRANSACTION;  
25 SELECT * FROM StudentEnrollments WHERE enrollment_id = 1 FOR UPDATE;  
26 UPDATE StudentEnrollments SET enrollment_date = '2024-07-04' WHERE enrollment_id = 1;  
27  
28 COMMIT;  
29 COMMIT;  
30 SELECT * FROM StudentEnrollments WHERE enrollment_id = 1;
```

Below the script, a table with the following data is displayed:

enrollment_id	student_name	course_id	enrollment_date
1	Ashish	CSE101	2024-07-01T00:00:00.000Z

The bottom of the image shows a Windows taskbar with the date and time '3:49 PM 9/27/2025'.

➤ **LEARNING OUTCOMES:**

1. Learned to enforce unique constraints to prevent duplicate student enrollments.
2. Understood row-level locking using SELECT FOR UPDATE to handle concurrent transactions.
3. Observed how transactions preserve Atomicity and Consistency in a multi-user environment.

4. Practiced handling blocked transactions and understanding isolation effects.
5. Gained hands-on experience with ACID principles in a practical enrollment scenario.